

浮點數行為實驗報告

Advanced C++ Homework 2

學號：113062331

姓名：林欣磊

繳交日期：2026/10/07

報告目的

透過可重現的程式、執行結果與組合語言，分析浮點型態的效能與精度、數學函式的實作、編譯器最佳化及十進位輸出行為。主要平台為 Intel 筆電的 WSL2 環境，另以 Apple M4 平台進行跨環境比較。

填寫方式

本文件為報告架構。請將待填欄位換成實測內容，插入截圖並完成分析；提交前刪除填寫提示。每題以實驗目的、方法、結果與原因分析呈現。題目細節與加分條件請以正式作業說明為準。

內容

Q0 實驗環境

Q1 不同浮點型態的效能與精度

Q2 數學函式實作

Q3 IEEE 754 與 fast math

Q4 浮點數輸出

Q5 跨執行環境比較

Q6 總結與額外浮點問題

Q7 AI 使用揭露

參考資料

Q0 實驗環境

0 1 主要與輔助平台

項目	主要平台（助手本機採集）	輔助平台（本人提供）
裝置	ASUS TUF Gaming F16 FX608JMR	MacBook Air（2025）
處理器	Intel Core i7-14650HX	Apple M4
架構／target	x86_64-linux-gnu	arm64-apple-darwin27.0.0
主機系統	Windows 11 家用版 25H2 Build 26200.9457	macOS Golden Gate 27.0.1 名稱與 build 待 sw_vers 核對
執行環境	WSL2／Ubuntu 26.04.1 LTS	macOS 原生環境
編譯器與版本	GCC 15.2.0 Ubuntu 15.2.0-16ubuntu1	Apple Clang 21.0.0 clang-2100.3.24.2
Q0 編譯參數	-std=c++20 -O2 -fno-fast-math -ffp-contract=off	尚未執行 Q0 型態程式
核心／執行緒	Windows：16／24 WSL 虛擬拓樸：12／24	待採集
記憶體	實體 32 GiB；WSL 約 15 GiB WSL swap 4 GiB	24 GB（本人提供）
WSL／kernel	WSL 2.7.3.0 6.6.114.1-microsoft-standard-WSL2	不適用／Darwin kernel 待採集
函式庫	glibc 2.43-2ubuntu2.4 libstdc++6 16-20260322-1ubuntu1	實際連結版本待採集
電源與採集日期	平衡；AC 電源；電池 100% 2026/10/04（Asia/Taipei）	資訊提供：2026/10/04 電源狀態待採集

WSL 拓樸為虛擬環境資訊。正式執行改用 GNU GCC 16.2.0：WSL 安裝於使用者目錄，Mac 為本人提供的 Homebrew 版本，完整輸出待核對。原有 compiler 保留，上表與型態表為先前採集；新版紀錄待本人重跑。

```
Windows PowerShell :  
./experiments/q0_environment/collect_windows.ps1  
WSL Ubuntu（專案根目錄）：  
bash experiments/q0_environment/collect_linux.sh results/q0-wsl  
Mac（專案根目錄）：  
bash experiments/q0_environment/collect_mac.sh
```

【截圖位置】0 1 CPU 作業系統與編譯器資訊
插入圖片後補上圖說，說明圖片支持的觀察。

02 浮點型態資訊

下表為 WSL GCC 15.2.0 的實際輸出。sizeof 表示儲存空間；long double 的 16 bytes 不等於 128-bit 有效精度。Mac 型態特性尚待實測。

型態	sizeof	digits	digits10	max_digits10	is_iec559
float	4	24	6	9	true
double	8	53	15	17	true
long double	16	64	18	21	true

WSL 補充輸出：radix=2；alignof 為 4／8／16 bytes；epsilon 為 2^{-23} ／ 2^{-52} ／ 2^{-63} ；FLT_EVAL_METHOD=0；捨入模式為 FE_TONEAREST；fast-math 關閉。編譯器支援 _float128 擴充，但尚未測試其運算。

Q0 重現指令（WSL，專案根目錄）：

```
g++ -std=c++20 -O2 -fno-fast-math -ffp-contract=off experiments/q0_environment/main.cpp -o results/q0-wsl/q0_environment
./results/q0-wsl/q0_environment
```

原始紀錄：results/q0-windows/hardware.json、results/q0-windows/wsl-power.txt、results/q0-wsl/environment.txt。環境採集與型態程式由助手執行；本人仍須重跑、觀察並加入截圖。CPU 即時頻率、溫度與 Mac 完整採集尚待補充。

【截圖位置】02 型態資訊程式輸出

插入圖片後補上圖說，說明圖片支持的觀察。

統一執行方式：Windows 先跑 collect_windows.ps1；WSL 專案根目錄執行 bash scripts/run_all.sh。每次自動建立 results/linux-日期時間，保存所有原始紀錄、命令、flags、來源雜湊與 assembly。各題結果及截圖尚待本人完成，實驗手冊見 docs/runbook.md。

Q1 不同浮點型態的效能與精度

1 1 實驗目的

使用相同演算法比較 float、double 與 long double 的執行時間及數值誤差。若另測 `_float128`，需記錄支援工具與額外編譯參數。

1 2 效能測試方法

準備測量 float、double、long double 的 scalar 加減、乘法及除法。每個 kernel 分為一條相依鏈與四條獨立鏈，每迭代分別為 2 與 8 個算術運算。初始值由執行期 seed 決定，`factor=1+1/1024`，配對運算維持值域。預設每組 2 次暖機（每次最多 100000 迭代）、7 次測量（每次 5000000 迭代）；使用 `steady_clock`，保留原始秒數、每運算時間、中位數與最小／最大值，輪換型態順序。

待測 kernel 獨立編譯，關閉 LTO、fast-math、FMA contraction 與自動向量化；`noinline` 與迴圈前後的 `optimization barrier` 防止跨函式簡化，每回合輸出 `checksum`。正式測量前檢查反組譯中仍有算術迴圈與分支。時間包含呼叫、少量參數準備、迴圈控制及收尾 `checksum` 成本；每運算時間不直接等於單條指令硬體 `latency`。

在 WSL Ubuntu 專案根目錄執行：
`bash experiments/q1_benchmark/build.sh`
`./build/q1-scalar/q1_benchmark --check`
`./build/q1-scalar/q1_benchmark --iterations 5000000 --repeats 7 --warmup 2 --seed 20261004`
`objdump -d -C build/q1-scalar/kernels.o`
編譯旗標：`-std=c++20 -O3 -Wall -Wextra -fno-fast-math -ffp-contract=off -fno-lto -fno-tree-vectorize`

1 3 效能結果

助手已用 GCC 16.2.0 完成全部建置、入口檢查及一輪真實 benchmark／精度驗證。原始樣本保存於 `results/linux-20261004-032643-assistant/`，明確標示操作者；以下本人結果表與截圖仍待親自執行後填入。

型態	Run 1	Run 2	Run 3	中位數
float	待填	待填	待填	待填
double	待填	待填	待填	待填
long double	待填	待填	待填	待填

若測量超過三次，保留完整原始紀錄，並於本表呈現摘要。

【截圖位置】1 1 Benchmark 原始輸出
插入圖片後補上圖說，說明圖片支持的觀察。

1 4 結果分析

【待填】解釋測試是在量測相依鏈的延遲、獨立運算的吞吐量，或整個演算法的時間。記錄是否向量化，避免把不同執行方式全歸因於型態精度。

Q1 精度與硬體分析

1 5 精度測試設計

精度測試使用 10000 組 $[2^{24}, 1, -2^{24}]$ 與 $[2^{53}, 1, -2^{53}]$ ，分別正序與反序加總。所有輸入皆可精確表示；實數精確答案為整數 10000，且三種型態都能精確表示。程式報告結果、絕對誤差與相對誤差，不假定 long double 為真值。執行：`./build/q1-scalar/q1_benchmark --precision-only`。

絕對誤差 = |實測值 - 參考值|
相對誤差 = 絕對誤差 / |參考值|，參考值為零時不使用此定義。

型態	計算結果	絕對誤差	相對誤差
float	待填	待填	待填
double	待填	待填	待填
long double	待填	待填	待填

【截圖位置】1 2 精度實驗輸出
插入圖片後補上圖說，說明圖片支持的觀察。

1 6 型態表示與指令

【待填】以實測 numeric_limits 資訊判斷有效精度，再用平台 ABI 或工具文件佐證表示方式。分別說明 float、double、long double。

【截圖位置】1 3 各型態運算核心的組合語言
插入圖片後補上圖說，說明圖片支持的觀察。

【待填】列出實際觀察到的指令或函式呼叫，說明它們如何支持效能差異的解釋。

1 7 小結

【待填】用一段文字總結本機測得的效能與精度取舍，並指出測試範圍。

Q2 數學函式實作

2 1 函式與規格

本節研究 `std::sqrt`，比較預設 `std`／`builtin` 路徑與強制 `libm symbol` 路徑，並測試三種型態及特殊值。

C++20 標準浮點 `overload` 分別回傳 `float`、`double`、`long double`；整數呼叫等效使用 `double`，本程式已用 `static_assert` 核對標頭。`sqrt` 計算非負平方根；負值 `domain error` 的 `errno`／`exception` 行為依 `math_errhandling` 與本機實測判斷。[1][5]

2 2 實驗方法

```
#include <cmath>
// experiments/q2_cmath/sqrt_paths.cpp
float sqrt_path(float x) { return std::sqrt(x); }
double sqrt_path(double x) { return std::sqrt(x); }
long double sqrt_path(long double x) { return std::sqrt(x); }
// 實際程式另有 noinline；libm 版使用 volatile function pointer。
```

在專案根目錄：`bash scripts/build_all.sh`
`./build/all/q2_cmath 2`
`./build/all/q2_libm 2`
旗標：`-O2 -fno-fast-math -frounding-math -ffp-contract=off -fno-lto`
`libm` 版另加 `-DHW2_FORCE_LIBM=1`；完整命令記錄在 `build.txt`。

2 3 本機實作路徑

研究路徑為 C++ `overload`、`compiler builtin`／`libm`、硬體指令或軟體演算法。查閱的 `glibc generic double sqrt` 以 `exponent`／`mantissa` 縮放、表格 `reciprocal sqrt` 初值、多項式修正與誤差補償處理捨入；另有直接 `builtin` 分支。此 `master` 來源尚未與 `Ubuntu` 套件精確匹配，正式路徑須用批次保存的反組譯驗證，不能將 `generic` 分支直接認定為本機實作。[5]

【截圖位置】2 1 `sqrt` 的組合語言與分支
插入圖片後補上圖說，說明圖片支持的觀察。

2 4 型態比較

型態	指令或函式呼叫	觀察與來源
float	待填	待填
double	待填	待填
long double	待填	待填

2 5 小結

【待填】說明本平台如何執行 `sqrt`，以及哪些結論僅適用於目前版本與編譯參數。

Q3 IEEE 754 與 fast math

3 1 實驗目的

以相同程式比較一般最佳化與 `-ffast-math`，找出輸出差異及造成差異的具體轉換。

GCC 16.2.0 的 `fast-math` 會放寬 `errno`、有限值、捨入模式與運算順序等語意，並允許 `associative`／`reciprocal` 類轉換。本實驗主要隔離加法重結合，另測 NaN 的有限值假設；兩版本同設 `-ffp-contract=off`，避免混入 FMA。[2]

3 2 程式與編譯方式

關鍵運算為 $(a+b)+c$ 、 $a+(b+c)$ 、 $x!=x$ 。a、b、c 從 argv 讀入，使用 1e16、-1e16、1；獨立 translation unit 與關閉 LTO 保留執行期計算。NaN 以 `binary64 quiet NaN bits` 建立，屬有限值假設的補充案例，與主要有限輸入反例分開分析。

```
bash scripts/build_all.sh
./build/all/q3_strict 1e16 -1e16 1
./build/all/q3_fast 1e16 -1e16 1
兩版本同用 -std=c++20 -O3 -fno-lto -ffp-contract=off；
只將 -fno-fast-math 換成 -ffast-math。
完整來源：experiments/q3_fast_math/main.cpp 與 operations.cpp。
```

3 3 實驗結果

模式	輸出	關鍵指令或轉換
一般模式	待填	待填
fast math	待填	待填

【截圖位置】3 1 兩種模式輸出
插入圖片後補上圖說，說明圖片支持的觀察。

【截圖位置】3 2 與 3 3 兩種模式關鍵 assembly
插入圖片後補上圖說，說明圖片支持的觀察。

3 4 原因分析與小結

【待填】從原始運算順序、編譯器轉換、各步捨入到最終結果說明差異。根據實驗列出適用情境與需要保留的數值保證。

Q4 浮點數輸出

4 1 問題

printf("%.9f", x) 與 fixed 搭配 setprecision(9)，是否能精確描述為將原本的十進位數四捨五入到小數點後第九位？

4 2 反例與驗證

選用 $1/1024=0.0009765625$ ，這是可精確表示的二進位值，也是 9 位小數的精確中點，介於 0.000976562 與 0.000976563。比較正負值、nextafter 鄰居及四種 rounding mode；每筆記錄 hexfloat、高精度儲存值、printf 與 iostream。結果仍待本人驗證。

```
// 完整程式：experiments/q4_formatting/main.cpp
double tie = std::ldexp(1.0, -10);
std::fesetround(FE_TONEAREST);
printf("%.9f\n", tie);
std::cout << std::fixed << std::setprecision(9) << tie;
// 另測負值、nextafter 與 upward/downward/towardzero。
// 編譯：bash scripts/build_all.sh
// 執行：./build/all/q4_formatting 0.0009765625
```

觀察項目	結果
原始十進位字面值	待填
實際儲存值或十六進位表示	待填
printf 輸出	待填
iostream 輸出	待填
捨入模式與環境	待填

【截圖位置】4 1 反例輸出與儲存值
插入圖片後補上圖說，說明圖片支持的觀察。

4 3 結論

【待填】精確回答原敘述，說明二進位表示、十進位中點及 ties 的處理。若討論 rounding mode，提供實測或文件依據。

Q5 跨執行環境比較

5 1 實驗條件

本節準備兩個候選。A 以同一 `binary`、相同數值輸入比較捨入環境；B 以 `GNU GCC 16.2.0` 比較兩平台 `long double` ABI。Mac 版本依本人提供，兩邊完整版本、來源、`flags` 與實際輸出待核對，不能只憑同版安裝即宣稱完成。

控制條件	Intel 平台	M4 平台
原始碼版本或雜湊	待填	待填
編譯器名稱與完整版本	待填	待填
編譯參數	待填	待填
OS 與函式庫	待填	待填
浮點環境與 ABI	待填	待填

5 2 程式与結果

候選 A：`add_double(1.0, 2-53)`，以 `HW2_ROUNDING` 選 `mode`；候選 B：`nearest` 下對 `long double` 計算 $(2^{53}+1)-2^{53}$ ，加法結果明確存回型態。獨立函式、關閉 `LTO`、`-frounding-math` 保留執行期運算；本案例不使用越界、未初始化值、資料競爭等一般 `C++ UB`。結果與條件仍待本人採集。[2]

【截圖位置】5 1 Intel 平台輸出
插入圖片後補上圖說，說明圖片支持的觀察。

【截圖位置】5 2 Apple M4 平台輸出
插入圖片後補上圖說，說明圖片支持的觀察。

5 3 原因分析

【待填】以具體證據判斷差異來自表示格式、指令、ABI、函式庫或浮點環境。若同時改變多個條件，說明哪些原因尚未被隔離。

5 4 小結

【待填】指出是否滿足加分條件，以及證據支持的結論。

Q6 總結與額外浮點問題

6 1 實驗總結

【待填】整合型態精度與效能、數學函式、最佳化及輸出的實驗發現。每項結論連結到本報告的結果或圖。

6 2 開發中的使用原則

【待填】說明如何依誤差容忍度選型態、處理運算順序與累積誤差、驗證最佳化。浮點比較應依用途選擇精確比較或適當誤差準則，不宜一律禁止 ==。

6 3 額外問題與實驗

額外主題為 gradual underflow。觀察接近零時 subnormal 的保留與精度，以及精確 subnormal 和 tiny/inexact 結果是否有不同 exception flags；這與 Q1 的加總案例分開討論。

三組算式： $\text{min_normal} \times 0.5$ ； $\text{denorm_min} \times 0.5$ ； $\text{denorm_min} \times 1$ 。
每組記錄結果、hexfloat、fpclassify、FE_UNDERFLOW、FE_INEXACT。

程式：experiments/q6_extra/main.cpp 與 operations.cpp。

編譯：bash scripts/build_all.sh；執行：./build/all/q6_extra。

先設定 nearest，x86 另讀取 MXCSR 的 FTZ/DAZ，但不改設定。不能以 SSE 控制位解釋 x87 long double；本題不量測 subnormal 速度。

型態／案例	result / hexfloat	分類	UF / inexact
待本人執行	待填	待填	待填

【截圖位置】6 1 三種型態的 subnormal/underflow 輸出
包含 exception flags、rounding mode 與 x86 MXCSR。

6 4 結論

【待填】根據實測區分 subnormal 表示、捨入為零與 underflow/inexact flags，並整合 Q1~Q5 提出浮點運算使用準則。

Q7 AI 使用揭露與參考資料

7 1 AI 使用揭露

請依实际使用情况修改下列文字，特别是尚未完成的執行與驗證部分。

我使用 Codex 協助建立專案、撰寫程式與腳本、查閱來源及整理報告方法。助手執行 Q0 採集與建置檢查，後續依我授權安裝 WSL GCC 16.2.0，跑真實 benchmark／精度及 Q2～Q6 驗證，資料獨立標示 assistant。我提供 Mac 設備與 compiler 資訊；本人實際執行、觀察、截圖與驗證方式：【待填】。

工具	協助內容	本人驗證方式
ChatGPT	依本人實際使用補充	待填
Codex	程式、腳本、來源查閱、Q0 採集、報告方法	正式執行、觀察與截圖待補

參考資料

以下為待查證的來源類別。完成查閱後，填入文件名稱、版本或章節、網址與查閱日期，並在相關段落標示引用編號。

[1] C++ working draft c.math：https://eel.is/c++draft/c.math（現行草案；C++20 overload 另以編譯檢查）。[2] GCC 16.2.0 Optimize Options：https://gcc.gnu.org/onlinedocs/gcc-16.2.0/gcc/Optimize-Options.html[3] IEEE 754 正式標準／相關表示與例外條文：【待本人查閱】[4] Intel SDM：https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html（特定章節待詳讀）。[5] glibc sqrt manual 與 e_sqrt.c／e_sqrtl.c／wrapper：完整 URL 見專案 references/sources.md；master source 尚未匹配本機套件。[6] Apple arm64 ABI：https://developer.apple.com/documentation/xcode/writing-arm64-code-for-apple-platforms（long double 等同 double；Mac 實測待取得）。上述已查閱來源日期：2026/10/04；理論筆記見 docs/reference_notes.md。

繳交前確認

所有結果均有原始紀錄；圖片可讀且附圖說；編譯與執行指令完整；參考資料可追溯；AI 揭露符合實際使用情況；刪除待填欄位與填寫提示。